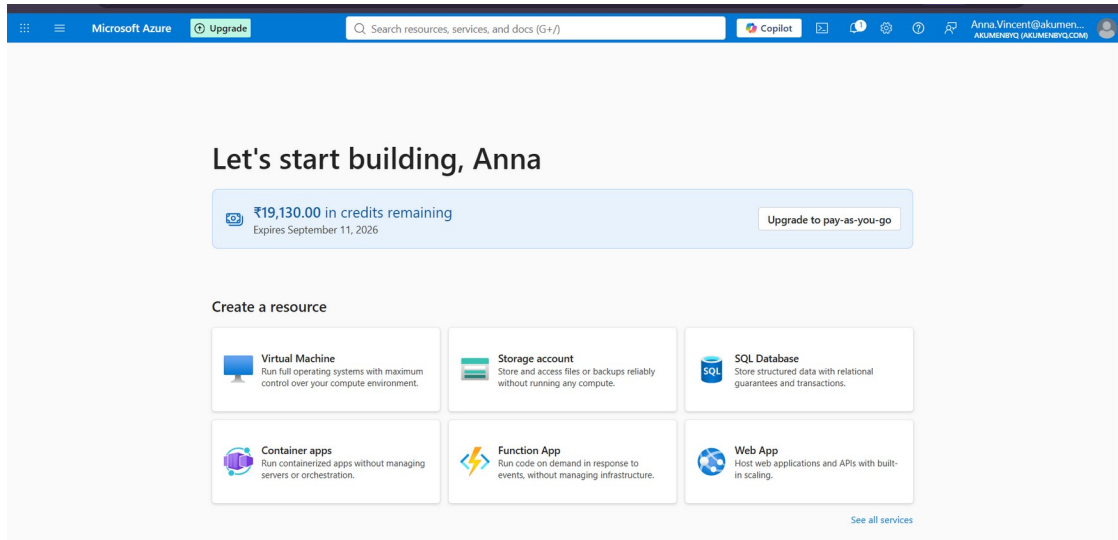


Azure Data Factory: Copy Activity Project

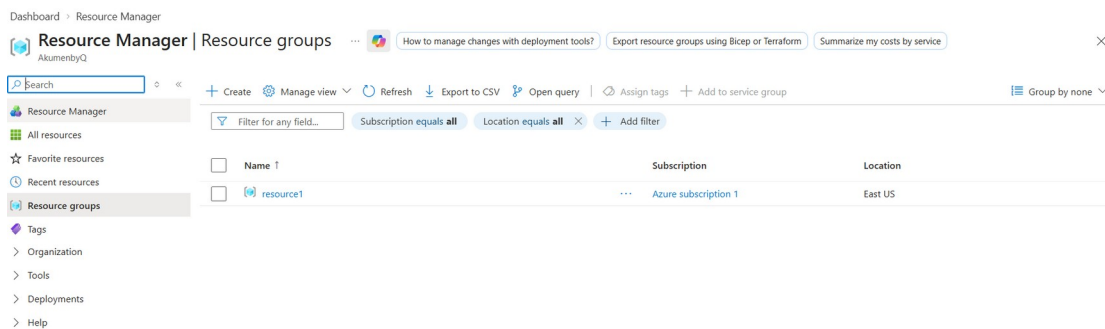
Step 1: Open the Azure Portal

This is just the Azure home page after logging in. From here you can create things like storage accounts and Data Factory.



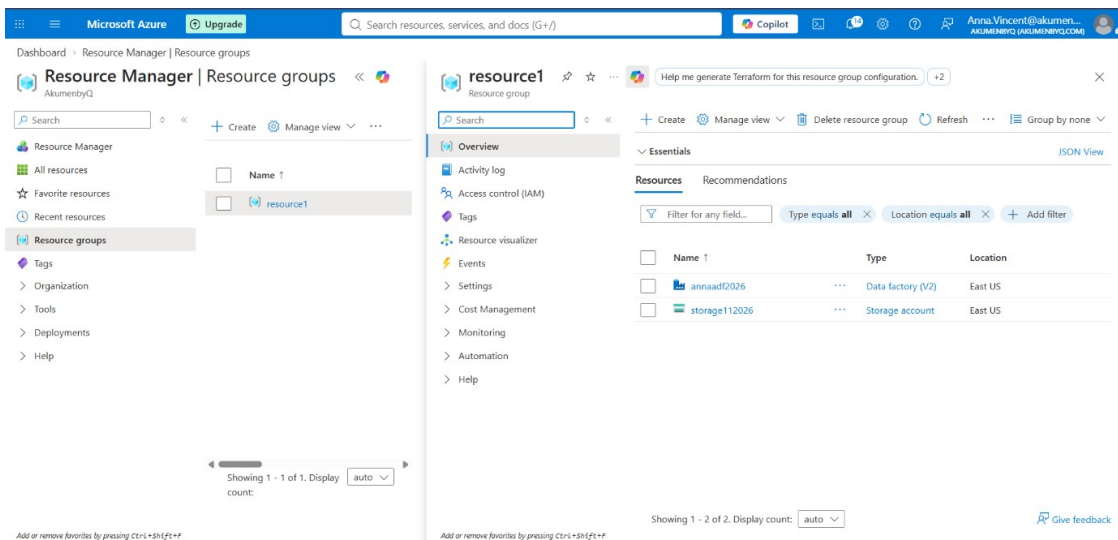
Step 2: Create a Resource Group

A resource group is like a folder that holds everything for this project together — the storage account and the Data Factory.



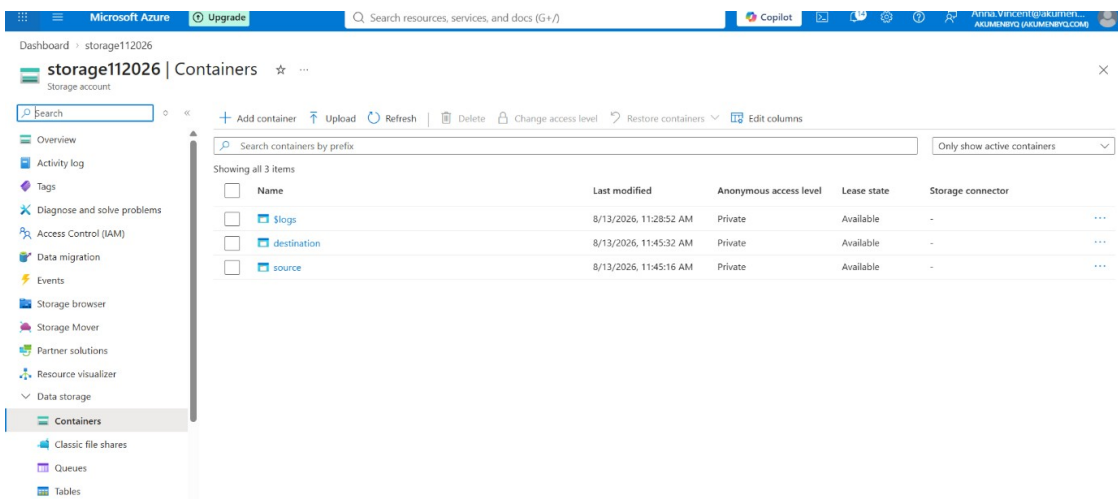
Step 3: Check What's Inside

Opening the resource group shows the two things we created: a Data Factory and a Storage account.



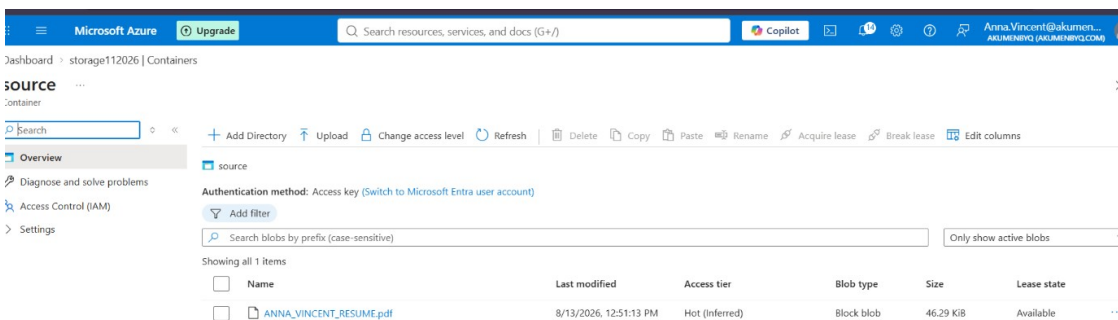
Step 4: Create Containers

Inside the storage account we have two containers: source and destination. Source holds the files we want to copy, destination is where they'll end up.



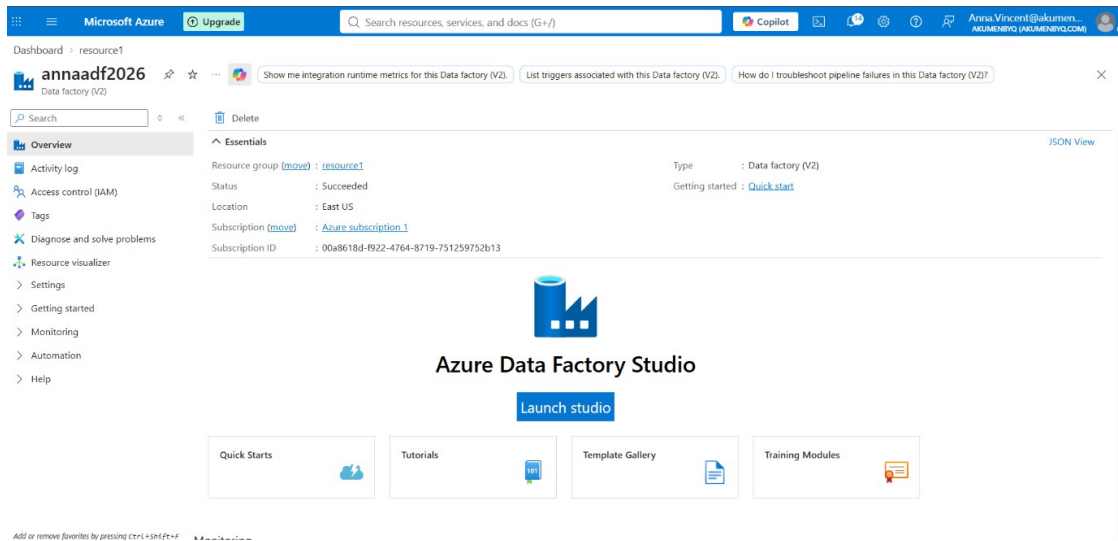
Step 5: Upload a File

A sample PDF file is uploaded to the source container. This is the file we'll copy first.



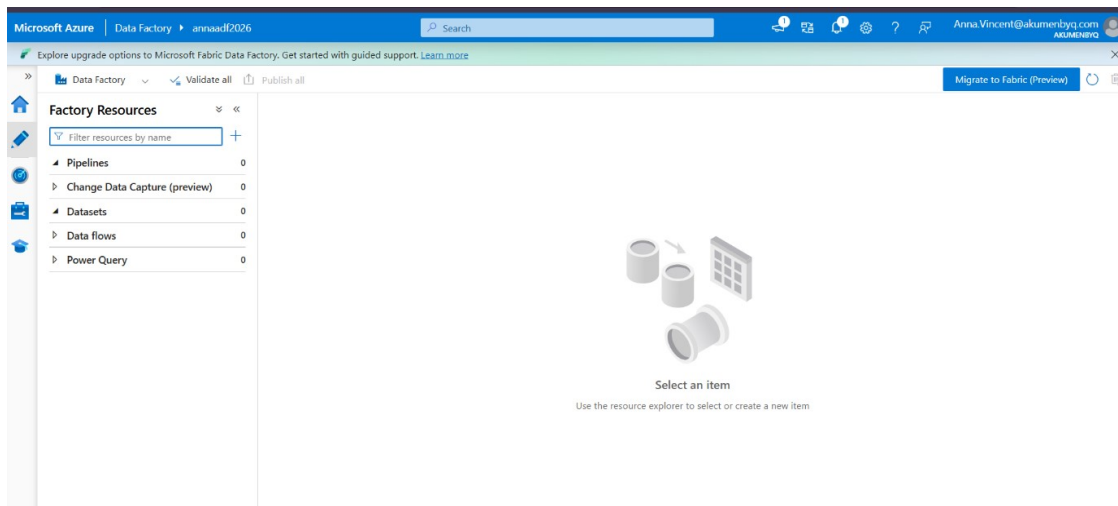
Step 6: Open Data Factory

This is the Data Factory we created. Clicking “Launch studio” takes us to the actual workspace where pipelines are built.



Step 7: Open Data Factory Studio

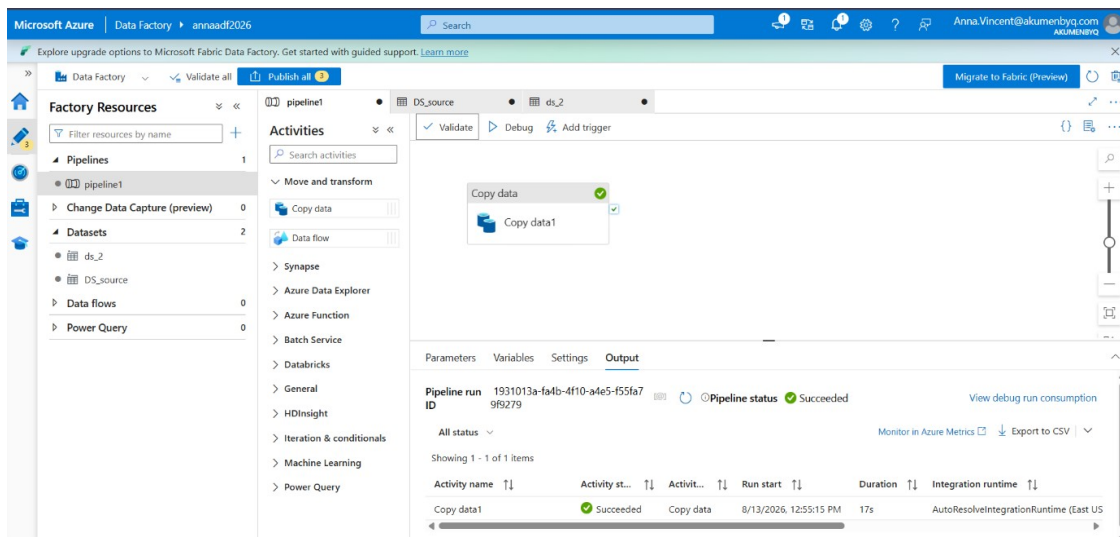
This is the Data Factory workspace. Pipelines, datasets, and everything else get created from the panel on the left.



Now the simplest case: copy a file from source to destination, no filters, nothing fancy.

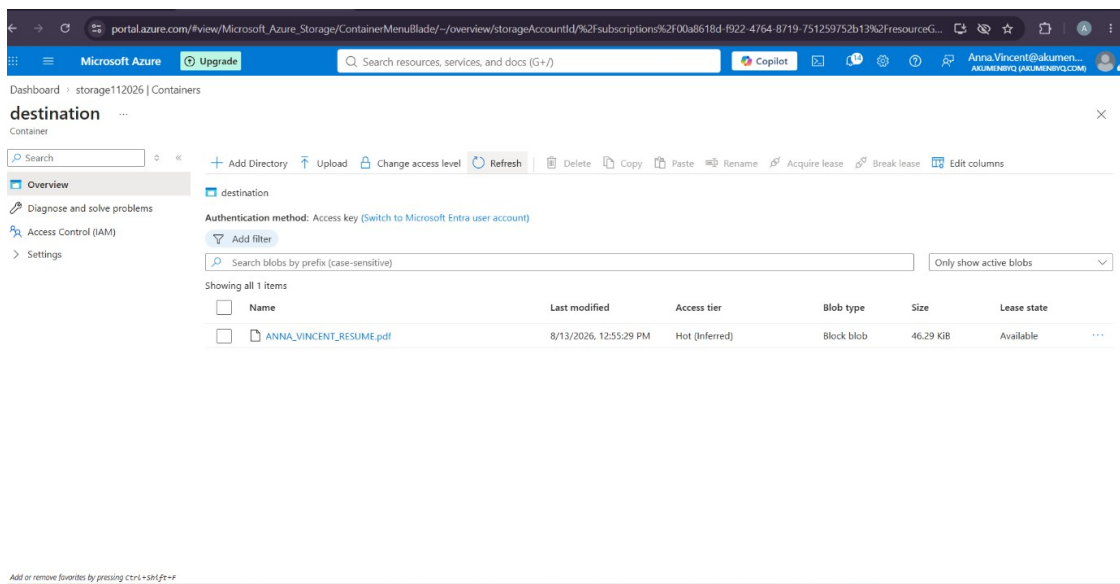
Step 8: Run the Pipeline

A Copy Data activity is added and pointed from source to destination, then run. It shows “Succeeded.”



Step 9: Check the Result

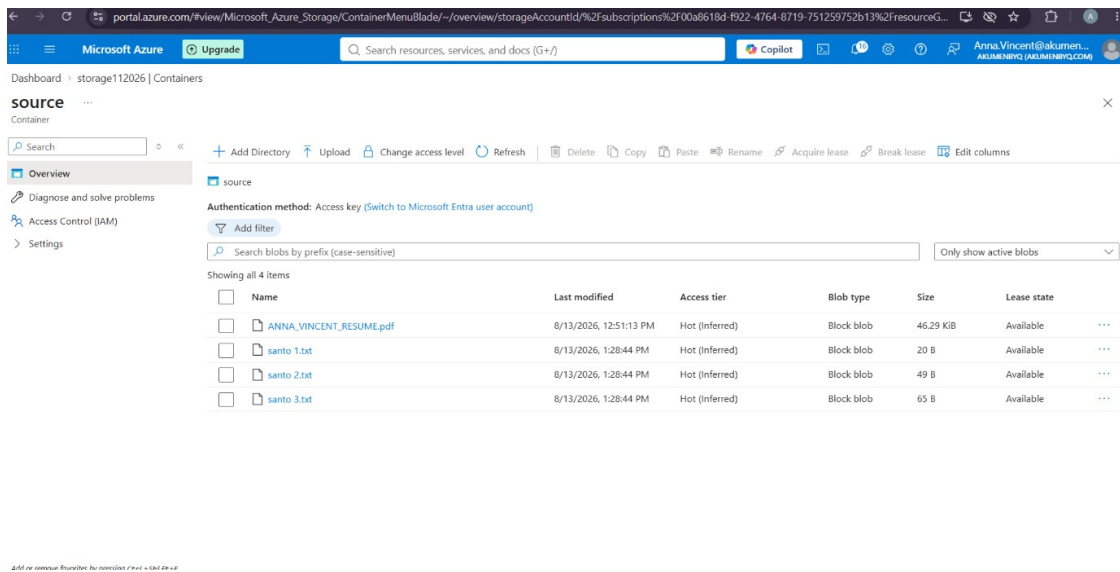
The destination container now has the file — the copy worked.



Next, only files whose names start with a certain word get copied.

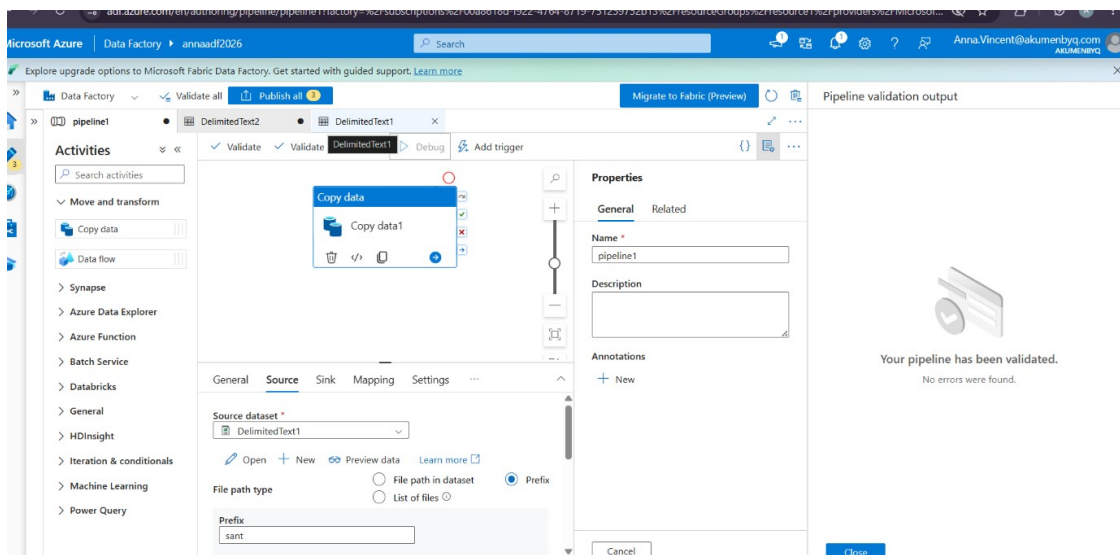
Step 10: Add More Files

Three text files (santo 1.txt, santo 2.txt, santo 3.txt) are added to source, along with the PDF already there.



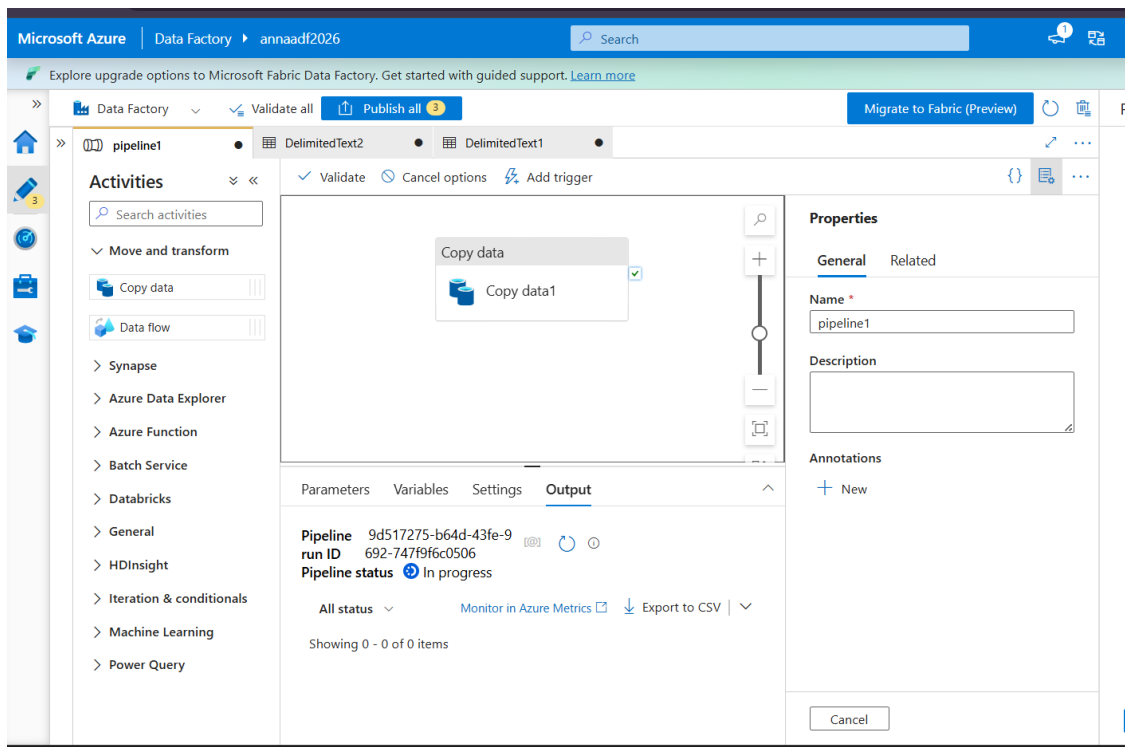
Step 11: Set the Prefix

In the source settings, “Prefix” is chosen and set to “sant”. This means only files starting with “sant” get copied — so just the three text files.



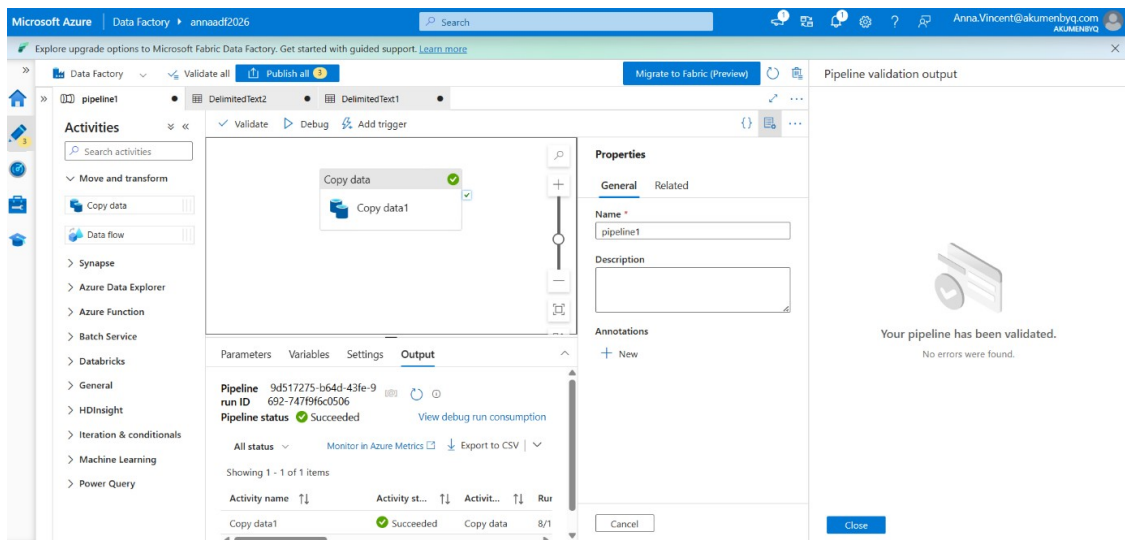
Step 12: Run It

The pipeline runs again. While it's working, the status shows “In progress.”



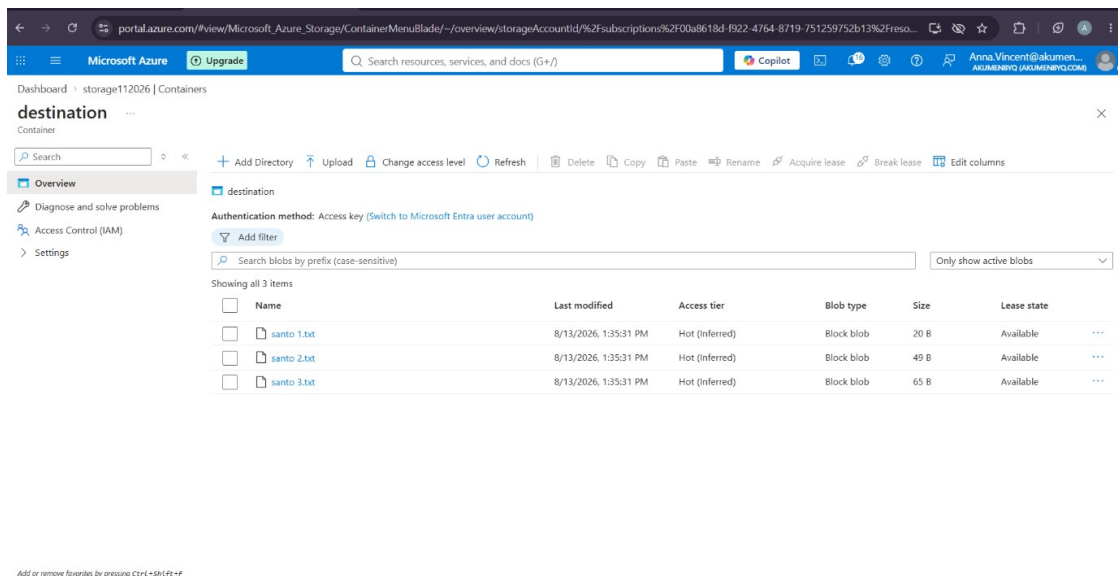
Step 13: It Finishes

The run completes and shows “Succeeded.”



Step 14: Check the Result

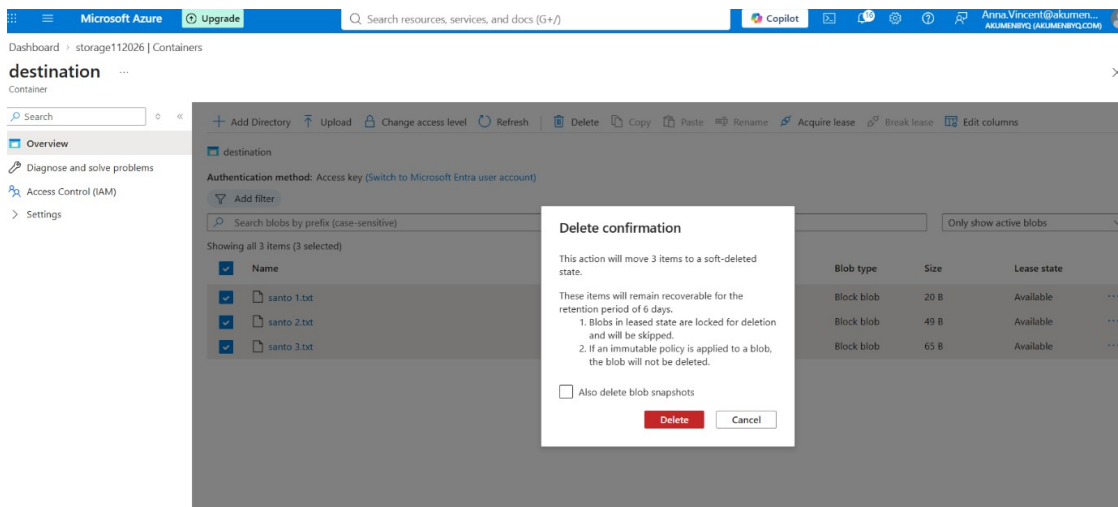
Only the three “santo” files show up in destination — the PDF was correctly left out.



Instead of a prefix, this uses a pattern — in this case, anything ending in .pdf.

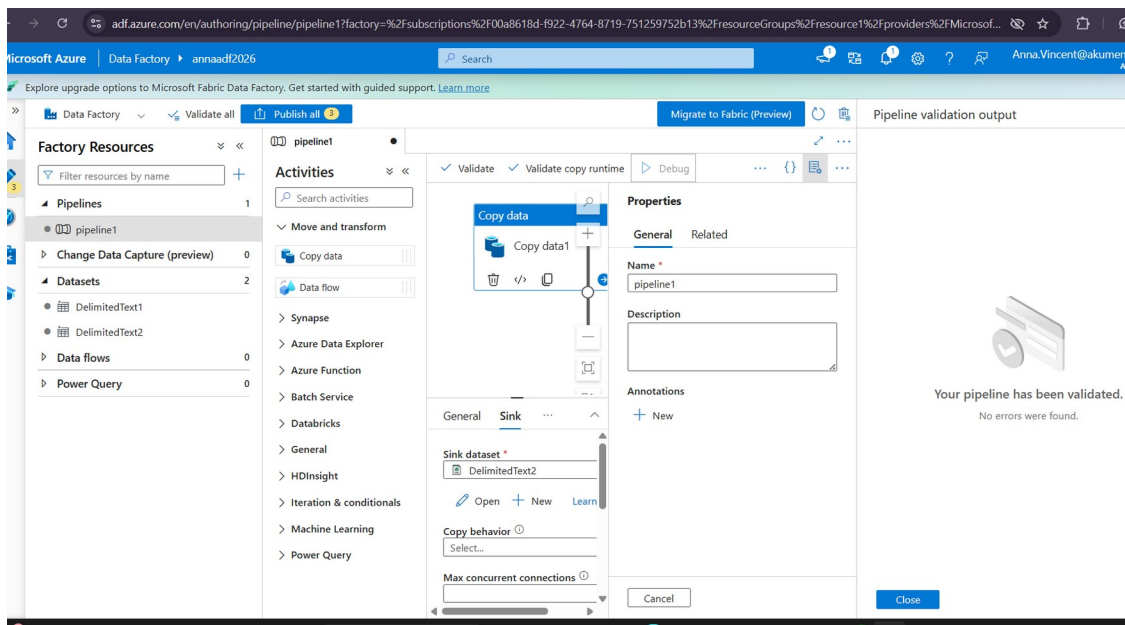
Step 15: Clear the Destination

The old files are deleted from destination first, so we can clearly see what the next run copies.



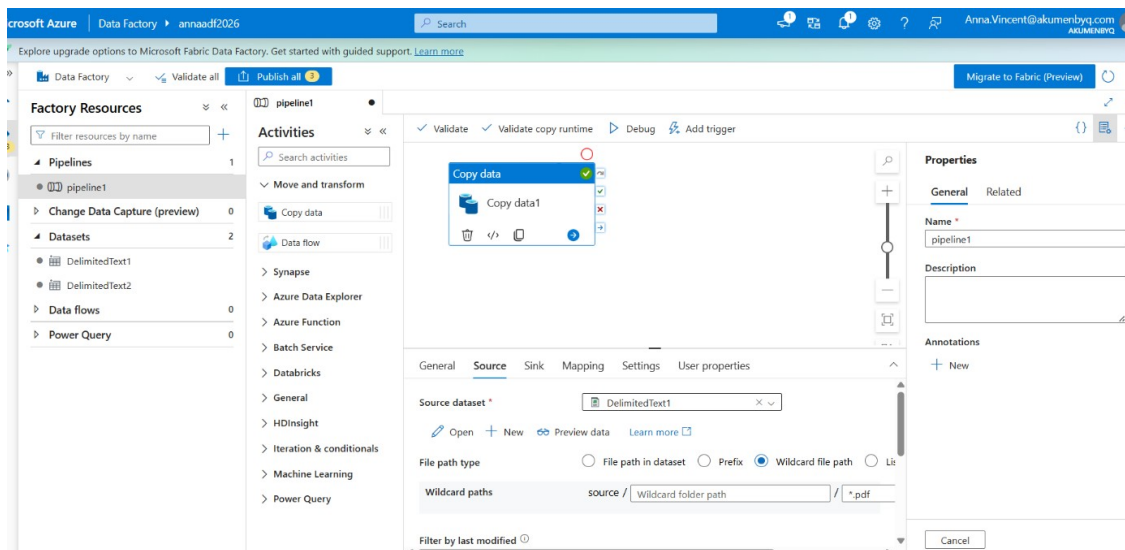
Step 16: Set the Destination Dataset

A new sink (destination) dataset is picked for this run.



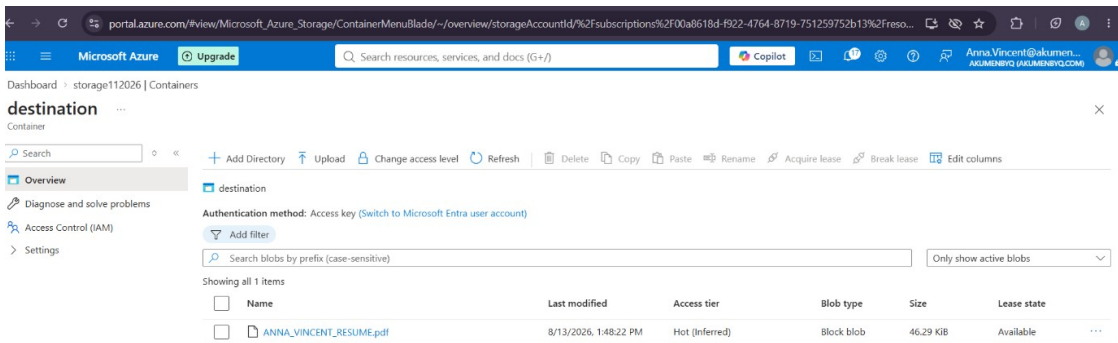
Step 17: Set the Wildcard Pattern

The source is set to “Wildcard file path” with the pattern *.pdf. This tells it to grab any file ending in .pdf, no matter the name.



Step 18: Check the Result

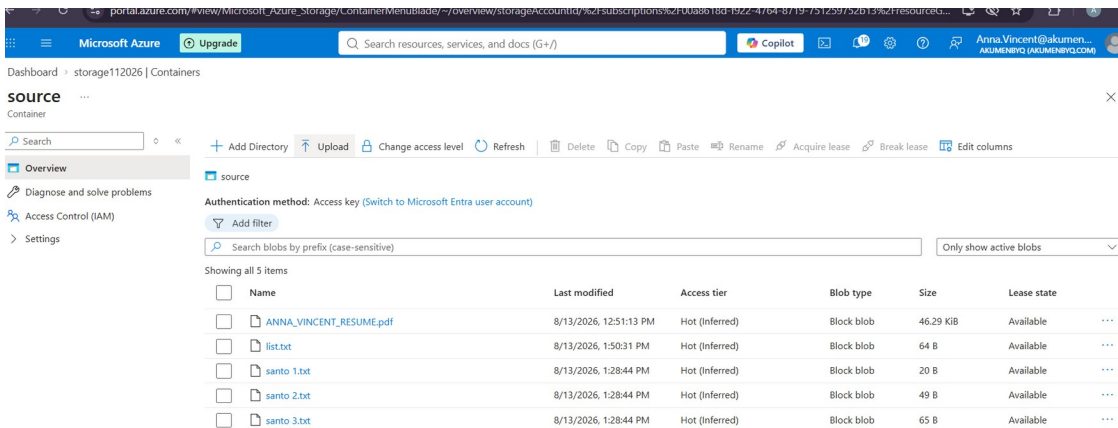
Only the PDF made it to destination — the text files were skipped.



This time we tell it exactly which files to copy by name, using a small text file as a list.

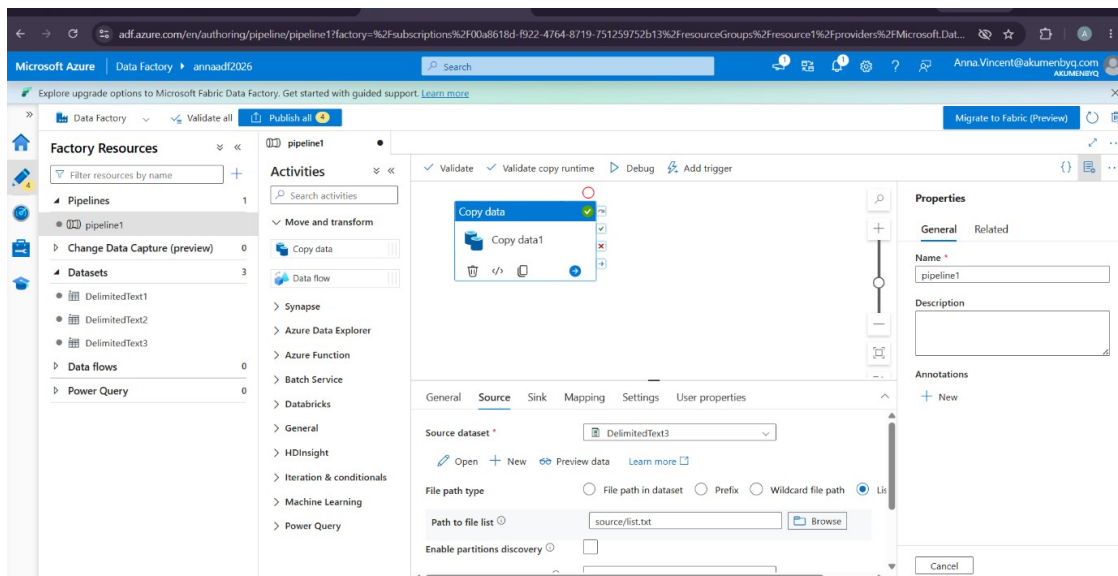
Step 19: Create the List File

A file called list.txt is added to source. It just contains the names of the files we want copied.



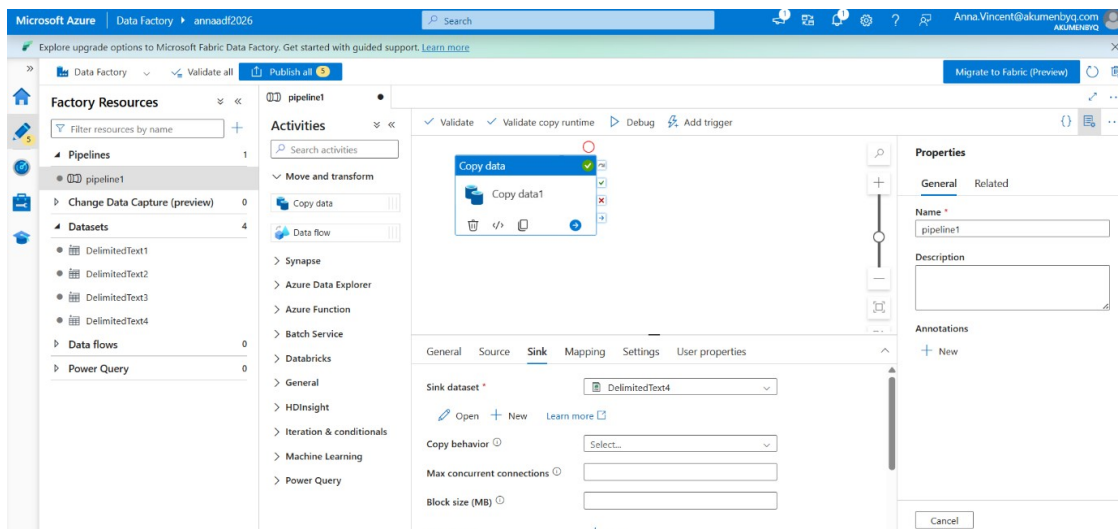
Step 20: Point to the List

The source is set to "List of files" and pointed at source/list.txt. It'll only copy what's named in that file.



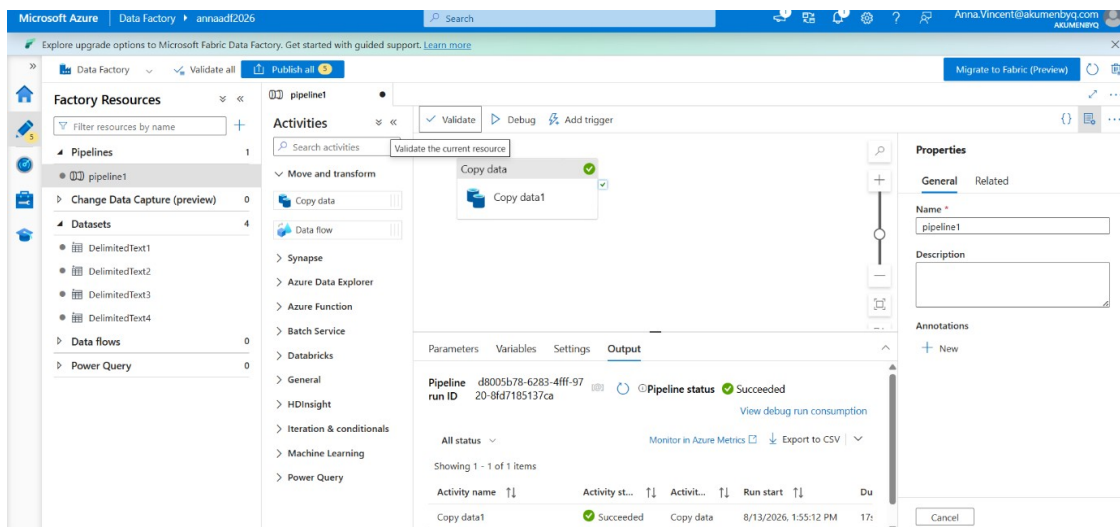
Step 21: Set the Destination Dataset

A new sink dataset is set up again for this run.



Step 22: Run the Pipeline

The pipeline runs and finishes with “Succeeded.”



Step 23: Check the Final Result

Destination now has exactly the files that were named in list.txt — nothing more, nothing less.

